

```

"""
Logic & Data Handling
Matrix operations and string manipulation -
WITHOUT using built-in libraries
(no numpy, no re, no math, no string
methods like .upper()/.split() etc.)
"""

#
=====
=====
# PART 1: MATRIX OPERATIONS
#
=====
=====

def create_matrix(rows, cols, fill=0):
    matrix = []
    for i in range(rows):
        row = []
        for j in range(cols):
            row.append(fill)
        matrix.append(row)
    return matrix

def print_matrix(matrix, name="Matrix"):
    print(name + ":")
    for row in matrix:
        line = ""

```

```

        for val in row:
            line += str(val) + "\t"
        print(line)
    print()

def matrix_add(A, B):
    rows = len(A)
    cols = len(A[0])
    result = create_matrix(rows, cols)
    for i in range(rows):
        for j in range(cols):
            result[i][j] = A[i][j] + B[i]
[j]
    return result

def matrix_subtract(A, B):
    rows = len(A)
    cols = len(A[0])
    result = create_matrix(rows, cols)
    for i in range(rows):
        for j in range(cols):
            result[i][j] = A[i][j] - B[i]
[j]
    return result

def matrix_multiply(A, B):
    rowsA = len(A)
    colsA = len(A[0])
    rowsB = len(B)
    colsB = len(B[0])

```

```

        if colsA != rowsB:
            raise ValueError("Cannot multiply:
incompatible dimensions")

    result = create_matrix(rowsA, colsB)
    for i in range(rowsA):
        for j in range(colsB):
            total = 0
            for k in range(colsA):
                total += A[i][k] * B[k][j]
            result[i][j] = total
    return result

def matrix_transpose(A):
    rows = len(A)
    cols = len(A[0])
    result = create_matrix(cols, rows)
    for i in range(rows):
        for j in range(cols):
            result[j][i] = A[i][j]
    return result

def matrix_scalar_multiply(A, scalar):
    rows = len(A)
    cols = len(A[0])
    result = create_matrix(rows, cols)
    for i in range(rows):
        for j in range(cols):
            result[i][j] = A[i][j] * scalar
    return result

```

```

def get_minor(matrix, row, col):
    minor = []
    for i in range(len(matrix)):
        if i == row:
            continue
        new_row = []
        for j in range(len(matrix[i])):
            if j == col:
                continue
            new_row.append(matrix[i][j])
        minor.append(new_row)
    return minor

def matrix_determinant(matrix):
    n = len(matrix)

    if n == 1:
        return matrix[0][0]

    if n == 2:
        return (matrix[0][0] * matrix[1]
[1]) - (matrix[0][1] * matrix[1][0])

    det = 0
    sign = 1
    for col in range(n):
        minor = get_minor(matrix, 0, col)
        det += sign * matrix[0][col] *
matrix_determinant(minor)
        sign = -sign
    return det

```

```

def matrix_cofactor(matrix):
    n = len(matrix)
    cof = create_matrix(n, n)
    for i in range(n):
        for j in range(n):
            minor = get_minor(matrix, i, j)
            sign = 1 if (i + j) % 2 == 0
            else -1
            cof[i][j] = sign *
matrix_determinant(minor)
        return cof

def matrix_adjoint(matrix):
    cof = matrix_cofactor(matrix)
    return matrix_transpose(cof)

def matrix_inverse(matrix):
    det = matrix_determinant(matrix)
    if det == 0:
        raise ValueError("Matrix is
singular, inverse does not exist")

    n = len(matrix)
    adj = matrix_adjoint(matrix)
    inverse = create_matrix(n, n)
    for i in range(n):
        for j in range(n):
            inverse[i][j] = adj[i][j] / det
    return inverse

def is_identity(matrix):

```

```

n = len(matrix)
for i in range(n):
    for j in range(n):
        expected = 1 if i == j else 0
        if matrix[i][j] != expected:
            return False
    return True

#
=====
# PART 2: STRING MANIPULATION (without
built-in str methods)
#
=====

def custom_length(s):
    count = 0
    for ch in s:
        count += 1
    return count

def custom_reverse(s):
    result = ""
    length = custom_length(s)
    for i in range(length - 1, -1, -1):
        result += s[i]
    return result

def is_palindrome(s):

```

```
        cleaned += custom_to_lower(ch)
    return cleaned ==
custom_reverse(cleaned)

def custom_to_upper(ch):
    if "a" <= ch <= "z":
        return chr(ord(ch) - 32)
    return ch

def custom_to_lower(ch):
    if "A" <= ch <= "Z":
        return chr(ord(ch) + 32)
    return ch

def custom_upper(s):
    result = ""
    for ch in s:
        result += custom_to_upper(ch)
    return result

def custom_lower(s):
    result = ""
    for ch in s:
        result += custom_to_lower(ch)
    return result
```

```
def custom_split(s, delimiter=" "):
    result = []
    current = ""
    for ch in s:
        if ch == delimiter:
            result.append(current)
            current = ""
        else:
            current += ch
    result.append(current)
    return result

def custom_join(parts, separator=" "):
    result = ""
    for i in range(len(parts)):
        result += parts[i]
        if i != len(parts) - 1:
            result += separator
    return result

def custom_strip(s):
    start = 0
    end = custom_length(s) - 1

    while start <= end and s[start] == " ":
        start += 1
    while end >= start and s[end] == " ":
        end -= 1

    result = ""
    for i in range(start, end + 1):
        result += s[i]
```



```

        return result

def custom_replace(s, old, new):
    result = ""
    i = 0
    old_len = custom_length(old)
    s_len = custom_length(s)

    while i < s_len:
        if i + old_len <= s_len and
s[i:i+old_len] == old:
            result += new
            i += old_len
        else:
            result += s[i]
            i += 1
    return result

def custom_count(s, target):
    count = 0
    target_len = custom_length(target)
    s_len = custom_length(s)

    for i in range(s_len - target_len + 1):
        if s[i:i+target_len] == target:
            count += 1
    return count

def custom_find(s, target):
    target_len = custom_length(target)
    s_len = custom_length(s)

```

```

        for i in range(s_len - target_len + 1):
            if s[i:i+target_len] == target:
                return i
        return -1

def word_frequency(s):
    s = custom_lower(custom_strip(s))
    words = custom_split(s, " ")
    freq = {}
    for word in words:
        cleaned = ""
        for ch in word:
            if ("a" <= ch <= "z") or ("0"
<= ch <= "9"):
                cleaned += ch
        if cleaned == "":
            continue
        if cleaned in freq:
            freq[cleaned] += 1
        else:
            freq[cleaned] = 1
    return freq

def is_anagram(s1, s2):
    s1 = custom_lower(custom_replace(s1, "
", ""))
    s2 = custom_lower(custom_replace(s2, "
", ""))

    if custom_length(s1) !=
custom_length(s2):

```

```

        return False

    freq1 = {}
    freq2 = {}

    for ch in s1:
        freq1[ch] = freq1.get(ch, 0) + 1
    for ch in s2:
        freq2[ch] = freq2.get(ch, 0) + 1

    return freq1 == freq2

def vowel_consonant_count(s):
    vowels = "aeiouAEIOU"
    vowel_count = 0
    consonant_count = 0
    for ch in s:
        if ("a" <= ch <= "z") or ("A" <= ch
<= "Z"):
            is_vowel = False
            for v in vowels:
                if ch == v:
                    is_vowel = True
                    break
            if is_vowel:
                vowel_count += 1
            else:
                consonant_count += 1
    return vowel_count, consonant_count

def caesar_cipher(s, shift):
    result = ""

```

```

        for ch in s:
            if "a" <= ch <= "z":
                new_pos = (ord(ch) - ord("a") +
shift) % 26
                result += chr(new_pos +
ord("a"))
            elif "A" <= ch <= "Z":
                new_pos = (ord(ch) - ord("A") +
shift) % 26
                result += chr(new_pos +
ord("A"))
            else:
                result += ch
        return result

#
=====
=====
# DEMO / TEST CASES
#
=====
=====

def run_matrix_demo():
    print("=" * 60)
    print("MATRIX OPERATIONS DEMO")
    print("=" * 60)

    A = [
        [1, 2, 3],
        [4, 5, 6],
        [7, 8, 10]
    ]

```

```

B = [
    [9, 8, 7],
    [6, 5, 4],
    [3, 2, 1]
]

print_matrix(A, "Matrix A")
print_matrix(B, "Matrix B")

print_matrix(matrix_add(A, B), "A + B")
print_matrix(matrix_subtract(A, B), "A
- B")
print_matrix(matrix_multiply(A, B), "A
* B")
print_matrix(matrix_transpose(A),
"Transpose of A")
print_matrix(matrix_scalar_multiply(A,
2), "A * 2")

det_A = matrix_determinant(A)
print("Determinant of A:", det_A)
print()

inv_A = matrix_inverse(A)
print_matrix(inv_A, "Inverse of A")

product = matrix_multiply(A, inv_A)
print_matrix(product, "A * A_inverse
(should be Identity)")
print("Is identity matrix:",
is_identity(
    [[round(val, 6) for val in row] for
row in product]

```

```

))

def run_string_demo():
    print("=" * 60)
    print("STRING MANIPULATION DEMO")
    print("=" * 60)

    text = " Hello World, Hello Python
World! "

    print("Original:", repr(text))
    print("Stripped:",
repr(custom_strip(text)))
    print("Length:", custom_length(text))
    print("Reversed:",
custom_reverse(text))
    print("Upper:", custom_upper(text))
    print("Lower:", custom_lower(text))
    print()

    print("Split by ' ':",
custom_split(custom_strip(text), " "))
    print("Join with '-':",
custom_join(["Hello", "World", "Python"],
"-"))
    print()

    print("Replace 'Hello' -> 'Hi':",
custom_replace(text, "Hello", "Hi"))
    print("Count of 'Hello':",
custom_count(text, "Hello"))
    print("Find 'Python':",
custom_find(text, "Python"))

```

```

print()

print("Word Frequency:",
word_frequency(text))
print()

s1 = "Listen"
s2 = "Silent"
print(f"Is '{s1}' an anagram of
'{s2}'?", is_anagram(s1, s2))
print()

palindrome_test = "A man a plan a canal
Panama"
print(f"Is '{palindrome_test}' a
palindrome?",
is_palindrome(palindrome_test))
print()

vc = vowel_consonant_count("Hello
World")
print("Vowels, Consonants in 'Hello
World':", vc)
print()

cipher = caesar_cipher("Hello World",
3)
print("Caesar Cipher (shift 3):",
cipher)
print("Decrypted back:",
caesar_cipher(cipher, -3))

if __name__ == "__main__":

```

```
run_matrix_demo()  
print()  
run_string_demo()
```